MATHEMATICAL GUIDE

Control, Optimization, and Learning for Quadruped Robots

PID, LQR, MPC, machine learning, neural networks, reinforcement learning, and PPO

A university-level tutorial with derivations, proofs, and a project-specific implementation roadmap

Prepared for the LittleMooMoo Quadruped Robot Project

Project snapshot reviewed August 11, 2026

How to use this guide

This document is written for an engineering undergraduate who already understands basic calculus and programming but wants the mathematical chain that connects feedback control, optimal control, numerical optimization, and modern reinforcement learning. The goal is not to memorize controller names. It is to see that all of them answer versions of the same question: given a dynamical system, incomplete measurements, physical limits, and a desired behavior, how should an action be chosen?

> **Core idea.** PID reacts to present and accumulated error. LQR chooses the best linear feedback law for a quadratic objective. MPC repeatedly solves a finite-horizon constrained optimal-control problem. A neural policy learns a nonlinear input-output map from data. PPO changes that policy using sampled rewards while limiting how far each update moves.

The treatment is rigorous where the underlying theorem is accessible at this level. Full proofs are provided for several central results: the finite-horizon LQR Riccati recursion, the LQR Lyapunov stability argument, the Bellman contraction, the backpropagation recursion, and the likelihood-ratio policy gradient. For results whose complete proofs require functional analysis or advanced nonlinear programming, the assumptions and proof logic are stated precisely and the missing technical machinery is identified.

Notation and conventions

Table 1. Symbols used throughout the guide.

Symbol	Meaning	Quadruped interpretation
x	state vector	body pose/velocity, joint states, estimator states
u	control input	joint target, velocity, torque, or contact force
y	measured output	IMU, encoder, current, or foot-contact signal
f(x,u)	nonlinear dynamics	rigid-body, actuator, contact, and ground dynamics
A, B, C	linearized state-space matrices	local model around standing or a gait phase
J(q)	leg Jacobian	maps joint velocity to foot velocity
pi_theta(a|s)	stochastic policy	distribution over 12 joint actions
gamma, lambda	discount and GAE parameters	time preference and bias-variance tuning

Contents

How to use this guide 2

Notation and conventions 2

Contents 2

Mathematical foundation 6
1.1 Dynamical systems 6
1.2 Equilibria and linearization 6
1.3 Stability and Lyapunov functions 6
1.4 Controllability and observability 7
1.5 Optimization and probability 7
PID control 8
2.1 Definition and physical meaning 8
2.2 Transfer-function derivation 8
2.3 Mechanical example and a stability condition 9
2.4 Second-order intuition 9
2.5 Discrete implementation 9
2.6 Anti-windup, saturation, and derivative kick 10
2.7 PID on the quadruped 10
Linear quadratic regulation 11
3.1 Problem statement 11
3.2 Finite-horizon Riccati derivation 11
3.3 Infinite-horizon LQR and stability 12
3.4 Continuous-time LQR 12
3.5 Tracking, integral augmentation, and LQG 12
3.6 Quadruped application 12
Model predictive control 13
4.1 Receding-horizon principle 13
4.2 Condensed quadratic program 13
4.3 Constraints and soft constraints 13
4.4 Recursive feasibility and stability 14
4.5 Nonlinear MPC 14
4.6 Centroidal quadruped dynamics 14
4.7 MPC with your position-controlled actuators 15
Machine learning 16
5.1 What is learned? 16
5.2 Learning paradigms 16
5.3 Gradient descent 16

5.4 Bias, variance, and regularization....17
5.5 System identification as the first ML task....17
Neural networks....18
6.1 A neuron and a multilayer network....18
6.2 Universal approximation: what it says and does not say....18
6.3 Backpropagation derivation....19
6.4 Losses and outputs....19
6.5 Neural networks in this project....19
Reinforcement learning....20
7.1 The Markov decision process....20
7.2 Value functions and Bellman equations....20
7.3 Bellman contraction proof....20
7.4 Policy gradients....20
7.5 Actor-critic and GAE....21
7.6 Designing a quadruped MDP....21
Proximal policy optimization....22
8.1 Why policy updates need restraint....22
8.2 Clipped surrogate objective....22
8.3 Complete actor-critic loss....23
8.4 PPO algorithm....23
8.5 Why PPO is used for locomotion....23
8.6 PPO safety and validation....23
How the methods fit together....24
Your current quadruped architecture....25
10.1 Inspected Python snapshot....25
10.2 Inspected C++ snapshot....26
10.3 The IMU kinematic idea....26
10.4 Why IMU alone is not a complete state estimator....26
A staged implementation roadmap....28
11.1 Stage 0: preserve the deterministic baseline....28
11.2 Stage 1: state estimator....28
11.3 Stage 2: low-bandwidth IMU posture PD....29
11.4 Stage 3: identify and test LQR....29

11.5 Stage 4: position-based MPC 29
11.6 Stage 5: torque or MIT-style low-level mode 29
11.7 Stage 6: PPO 29
11.8 Acceptance tests before each escalation 30
Worked quadruped examples 31
12.1 From joint motion to foot velocity 31
12.2 Static support-force distribution 31
12.3 Why a foot can move but the body does not 31
12.4 A small position-MPC design example 32
Proofs, assumptions, and common misconceptions 33
13.1 PID is not automatically stable 33
13.2 LQR optimality is model-relative 33
13.3 MPC is not magic prediction 33
13.4 Universal approximation is not universal safety 33
13.5 PPO clipping is not a hard action bound 33
13.6 More loop frequency is not always better 33
Exercises 34
14.1 Control theory 34
14.2 Machine learning and PPO 34
14.3 Project laboratories 34
14.4 Selected hints 35
Glossary and reference equations 36
15.1 Equation map 36
References 37
Closing perspective 37

CHAPTER 1

Mathematical foundation

Dynamics, linearization, stability, optimization, and probability

1.1 Dynamical systems

A controller does not act on a static formula. It acts on a state that evolves. In continuous time, a deterministic nonlinear system is written as

$$\dot{x}(t) = f(x(t), u(t)), y(t) = h(x(t), u(t)) \quad (1.1)$$

Here x(t) is the minimum information required to predict future motion when future inputs are known. A rigid quadruped state may contain body position p, body orientation R, body linear velocity v, body angular velocity omega, joint angles q, and joint velocities q-dot. In discrete time with sample period Delta t, the model becomes

$$x_{k+1} = f_d(x_k, u_k), y_k = h_d(x_k, u_k) \quad (1.2)$$

A real controller is therefore a sampled-data system: continuous mechanics evolve between discrete computer updates. Timing jitter, CAN latency, filtering, and zero-order hold are part of the plant seen by the controller, not merely software details.

1.2 Equilibria and linearization

An equilibrium (x_e,u_e) satisfies f(x_e,u_e)=0. Let delta x=x-x_e and delta u=u-u_e. A first-order Taylor expansion gives

$$\dot{\delta x} = A\delta x + B\delta u + \mathcal{O}(\|(\delta x, \delta u)\|^2) \quad (1.3)$$

$$A = \frac{\partial f}{\partial x}(x_e, u_e), B = \frac{\partial f}{\partial u}(x_e, u_e) \quad (1.4)$$

> **Why local controllers work.** When perturbations are small, the neglected second-order term is much smaller than the linear term. LQR and linear MPC can therefore regulate standing even though the complete robot is nonlinear. During large leg swings or contact changes, a single linearization becomes less trustworthy; gain scheduling, phase-dependent models, or nonlinear MPC may be required.

1.3 Stability and Lyapunov functions

An equilibrium is Lyapunov stable if every sufficiently small initial error remains small. It is asymptotically stable if the error also converges to zero. A scalar energy-like function V(x) proves stability when V(0)=0, V(x)>0 away from the origin, and its derivative decreases along trajectories:

$$\dot{V}(x) = \nabla V(x)^T f(x, u(x)) < 0, x \neq 0 \quad (1.5)$$

> **Lyapunov logic.** If V is positive, it behaves like a bowl around the equilibrium. If V-dot is negative, every trajectory moves downhill in that bowl. Because V cannot decrease below zero, trajectories

> cannot escape to larger energy levels. Under standard regularity and radial-unboundedness assumptions, this establishes asymptotic or global asymptotic stability.

1.4 Controllability and observability

For the linear discrete-time system x_{k+1}=Ax_k+Bu_k, the controllability matrix is

$$\mathcal{C} = [B \quad AB \quad A^2B \quad \cdots \quad A^{n-1}B] \quad (1.6)$$

The pair (A,B) is controllable when rank(C)=n. This means an appropriate finite input sequence can transfer the state between arbitrary points. Observability is the dual question: can the internal state be reconstructed from outputs? For y=Cx, define

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix}, \operatorname{rank}(\mathcal{O}) = n \quad (1.7)$$

A controller cannot stabilize an uncontrollable unstable mode, and an estimator cannot reconstruct an unobservable mode. This is why adding an IMU helps but does not automatically reveal global position: accelerometer bias and unobserved drift remain unless kinematic or external measurements constrain them.

1.5 Optimization and probability

Optimal control and learning repeatedly minimize an objective. A differentiable convex function has no false local minima. If its Hessian is positive semidefinite, then

$$\nabla^2 f(z) \succeq 0 \Longrightarrow f(y) \geq f(x) + \nabla f(x)^T (y - x) \quad (1.8)$$

A quadratic program (QP) has a quadratic objective and linear constraints:

$$\min_z \frac{1}{2} z^T H z + g^T z, l \leq Az \leq u, H \succeq 0 \quad (1.9)$$

Probability enters because sensors are noisy and learned policies sample actions. For random variable X, expectation is linear, covariance measures second-order uncertainty, and Bayes' rule updates belief after a measurement:

$$p(x \mid y) = \frac{p(y|x)p(x)}{p(y)} \quad (1.10)$$

CHAPTER 2

PID control

Error feedback in time and frequency domains

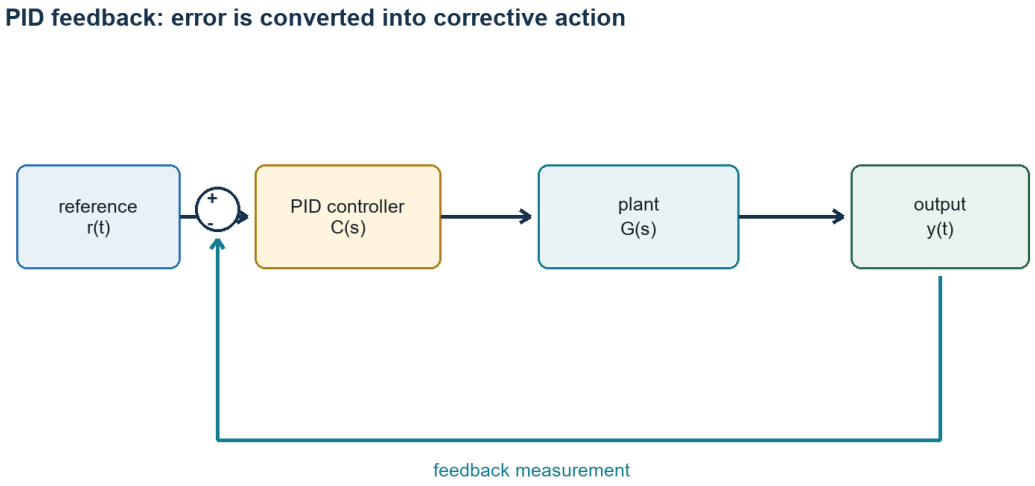


Figure 2.1. A PID controller operates on tracking error and closes the loop through measurement.

2.1 Definition and physical meaning

Let e(t)=r(t)-y(t). The ideal parallel PID law is

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt} \quad (2.1)$$

- Proportional action is a virtual spring: larger displacement error produces larger restoring effort.
- Integral action accumulates persistent error. It is useful against constant disturbances but stores energy and can wind up during saturation.
- Derivative action is a virtual damper: it opposes rapid error change and adds phase lead, but it amplifies high-frequency measurement noise.

In your motor position mode, the commonly called Kd is usually a velocity-feedback gain inside a cascaded position/velocity architecture. It is not automatically identical to differentiating a noisy position error in host software. The firmware, units, filters, and torque saturation determine the real closed-loop behavior.

2.2 Transfer-function derivation

With zero initial conditions, the Laplace transform converts differentiation to multiplication by s and integration to division by s. Thus

$$C(s) = K_p + \frac{K_i}{s} + K_d s \quad (2.2)$$

For plant G(s), unity negative feedback gives the closed-loop transfer function

$$T(s) = \frac{Y(s)}{R(s)} = \frac{C(s)G(s)}{1+C(s)G(s)} \quad (2.3)$$

The characteristic equation 1+C(s)G(s)=0 determines the closed-loop poles. Stability and transient response are properties of these poles, not of Kp or Kd in isolation.

2.3 Mechanical example and a stability condition

Consider a one-joint inertia with viscous damping and stiffness:

$$J\ddot{q} + b\dot{q} + kq = \tau \quad (2.4)$$

Apply PID torque tau=Kp(qr-q)+Ki integral(qr-q)dt-Kd q-dot for a constant reference. The closed-loop characteristic polynomial is

$$Js^3 + (b + K_d)s^2 + (k + K_p)s + K_i = 0 \quad (2.5)$$

For a cubic a3 s^3+a2 s^2+a1 s+a0, the Routh-Hurwitz criterion requires all coefficients positive and a2 a1>a3 a0. Therefore a sufficient and necessary condition for this ideal cubic is

$$J > 0,\ b + K_d > 0,\ k + K_p > 0,\ K_i > 0,\ (b + K_d)(k + K_p) > JK_i \quad (2.6)$$

> **Interpretation.** Increasing Ki without enough damping can destabilize the loop. Increasing Kp raises stiffness but also natural frequency, so fixed delay and unmodeled flexibilities become more dangerous. Increasing Kd can improve damping until sensor noise, quantization, delay, or actuator bandwidth dominates.

2.4 Second-order intuition

With PD control and k=0, the closed-loop denominator is Js^2+(b+Kd)s+Kp. Compare it with s^2+2 zeta omega_n s+omega_n^2:

$$\omega_n = \sqrt{\frac{K_p}{J}}, \zeta = \frac{b+K_d}{2\sqrt{JK_p}} \quad (2.7)$$

This equation explains a familiar tuning observation. Raising Kp alone raises natural frequency but decreases damping ratio when Kd stays fixed. To preserve zeta while increasing stiffness, Kd should scale approximately with the square root of Kp, assuming the simple model remains valid.

2.5 Discrete implementation

At sample period Delta t, a practical positional PID approximation is

$$I_k = sat(I_{k-1} + e_k \Delta t), D_k = \alpha D_{k-1} + (1-\alpha)\frac{e_k - e_{k-1}}{\Delta t} \quad (2.8)$$

$$u_k = sat(K_p e_k + K_i I_k + K_d D_k) \quad (2.9)$$

The derivative low-pass coefficient can be chosen from cutoff frequency fc:

$$\alpha = \frac{\tau_f}{\tau_f + \Delta t}, \tau_f = \frac{1}{2\pi f_c} \quad (2.10)$$

2.6 Anti-windup, saturation, and derivative kick

- Conditional integration: stop integrating when the actuator is saturated and the error would push farther into saturation.
- Back-calculation: add Kaw(usaturated-uunsaturated) to the integrator dynamics.
- Derivative on measurement: use -Kd dy/dt instead of Kd de/dt so a reference step does not create a derivative kick.
- Command shaping: use a smooth position/velocity/acceleration reference so the controller never sees an impossible instantaneous step.

2.7 PID on the quadruped

At the actuator level, joint position PD remains valuable even if higher layers use LQR, MPC, or PPO. A good hierarchy is: the high-level method produces physically consistent joint or foot references, while the firmware position loop rejects local motor disturbances. The high-level controller should not fight the low-level loop by injecting discontinuous targets faster than the actuator bandwidth.

$$\tau_{\text{cmd}} \approx K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}), |\tau_{\text{cmd}}| \leq \tau_{\max} \quad (2.11)$$

> **Project-specific caution.** Your 25:1 gearbox increases output torque and apparent stiffness but also increases reflected motor inertia, friction, and backlash sensitivity. Kp and Kd values are firmware-unit quantities, not universal SI gains. Compare settings only under the same firmware, gear ratio, filters, loop rate, and torque limit.

CHAPTER 3

Linear quadratic regulation

Optimal state feedback and the Riccati equation

3.1 Problem statement

LQR assumes a linear model and penalizes state deviation and control effort quadratically. For discrete dynamics

$$x_{k+1} = Ax_k + Bu_k \quad (3.1)$$

the finite-horizon objective is

$$J = x_N^T Q_N x_N + \sum_{k=0}^{N-1} (x_k^T Q x_k + u_k^T R u_k) \quad (3.2)$$

Assume Q and QN are positive semidefinite and R is positive definite. Q encodes what state errors are expensive; R encodes how reluctant the controller is to use input. Because R>0, the minimizer at each stage is unique.

3.2 Finite-horizon Riccati derivation

Dynamic programming says that an optimal policy minimizes the immediate cost plus the optimal future cost. Suppose the cost-to-go at k+1 is quadratic:

$$V_{k+1}(x) = x^T P_{k+1} x \quad (3.3)$$

Then

$$V_k(x) = min_u [x^T Q x + u^T R u + (Ax + Bu)^T P_{k+1} (Ax + Bu)] \quad (3.4)$$

Expand and collect the terms involving u:

$$V_k(x) = x^T (Q + A^T P_{k+1} A) x + \min_u [u^T (R + B^T P_{k+1} B) u + 2x^T A^T P_{k+1} B u] \quad (3.5)$$

Differentiate with respect to u and set the gradient to zero:

$$2(R + B^T P_{k+1} B) u + 2B^T P_{k+1} A x = 0 \quad (3.6)$$

$$u_k^\star = -K_k x_k, K_k = (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A \quad (3.7)$$

Substitution gives the backward Riccati recursion

$$P_k = Q + A^T P_{k+1} A - A^T P_{k+1} B (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A \quad (3.8)$$

> **Proof by backward induction.** At k=N, VN(x)=x^T QN x is quadratic. Equations 3.4-3.8 show that if Vk+1 is quadratic, then the minimizing policy is linear and Vk is quadratic. Therefore every stage has this form. This is not a guessed controller; it follows from Bellman's principle and completing the square.

3.3 Infinite-horizon LQR and stability

Under stabilizability of (A,B) and detectability of (Q^(1/2),A), Pk converges to a stabilizing solution P of the discrete algebraic Riccati equation. The constant feedback is u=-Kx.

$$P = Q + A^T PA - A^T PB(R + B^T PB)^{-1} B^T PA \quad (3.9)$$

$$K = (R + B^T PB)^{-1} B^T PA \quad (3.10)$$

Take V(x)=x^TPx. Under x+= (A-BK)x, use the Riccati equation to obtain

$$V(x_{k+1}) - V(x_k) = -x_k^T (Q + K^T RK) x_k \leq 0 \quad (3.11)$$

> **Stability proof.** If Q+K^TRK is positive definite, the decrease is strict for every nonzero state, so the origin is asymptotically stable. If it is semidefinite, detectability excludes an unpenalized unstable trajectory. Thus the optimal cost itself becomes a Lyapunov function.

3.4 Continuous-time LQR

For x-dot=Ax+Bu and cost integral from zero to infinity of x^TQx+u^TRu, assume V=x^TPx. The Hamilton-Jacobi-Bellman equation gives

$$A^T P + PA - PBR^{-1} B^T P + Q = 0 \quad (3.12)$$

$$u^\star = -R^{-1} B^T Px \quad (3.13)$$

This is the continuous algebraic Riccati equation. The same geometry appears: state error is converted to a costate 2Px, then B^T maps that value gradient into the input directions.

3.5 Tracking, integral augmentation, and LQG

Regulation drives x to zero. Tracking drives error x-xr to zero. For constant or slowly varying references, shift the coordinates or augment the state with the integral of output error:

$$\xi_{k+1} = \xi_k + (r_k - Cx_k)\Delta t, x_a = \begin{bmatrix} x \\ \xi \end{bmatrix} \quad (3.14)$$

If x is not measured, a Kalman filter can estimate it. LQG combines LQR and Kalman filtering. For an exactly linear Gaussian model, the separation principle says the controller and estimator can be designed independently, then connected as u=-K x-hat. This elegance does not remove modeling error; it states an optimality result under specific assumptions.

3.6 Quadruped application

For standing posture control, a small state could be x=[roll, pitch, roll-rate, pitch-rate]^T. Inputs could be commanded differential foot-height offsets or body-angle reference corrections rather than torques. Identify A and B experimentally around standing, then solve LQR. This creates a disciplined multi-input version of IMU PD: cross-coupling between roll and pitch is included automatically. One fixed gain remains local; when the support pattern changes during a trot, phase-scheduled LQR, periodic LQR, or MPC becomes more appropriate.

CHAPTER 4

Model predictive control

Online optimization with dynamics and constraints

4.1 Receding-horizon principle

At time k, MPC measures or estimates xk, predicts the system over N steps, chooses an input sequence, applies only the first input, then repeats from the next measured state. This repeated re-planning is the receding-horizon principle.

$$\min_{x_{0:N},u_{0:N-1}} \|x_N - x_r\|_P^2 + \sum_{i=0}^{N-1}\left(\|x_i - x_r\|_Q^2 + \|u_i - u_r\|_R^2\right) \quad (4.1)$$

$$x_{i+1} = Ax_i + Bu_i, x_i \in \mathcal{X}, u_i \in \mathcal{U}, x_0 = \hat{x}_k \quad (4.2)$$

LQR and unconstrained linear MPC are close relatives. With a sufficiently long horizon and a terminal cost equal to the Riccati solution, unconstrained MPC returns the LQR feedback. MPC becomes especially useful when constraints or planned contact changes are central.

4.2 Condensed quadratic program

Stack predicted states and inputs:

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}, U = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N-1} \end{bmatrix} \quad (4.3)$$

Repeated substitution of the dynamics gives

$$X = \mathcal{A}x_0 + \mathcal{B}U \quad (4.4)$$

$$\mathcal{A} = \begin{bmatrix} A \\ A^2 \\ \vdots \\ A^N \end{bmatrix}, \mathcal{B} = \begin{bmatrix} B & 0 & \cdots & 0 \\ AB & B & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A^{N-1}B & A^{N-2}B & \cdots & B \end{bmatrix} \quad (4.5)$$

Substitute Equation 4.4 into the quadratic cost. Terms independent of U can be discarded during minimization, yielding

$$\min_U \frac{1}{2}U^T HU + g(x_0)^T U, l(x_0) \le GU \le u(x_0) \quad (4.6)$$

When H is positive semidefinite and constraints are linear, this is a convex QP. Warm starting from the previous solution and caching matrix factorizations make real-time repeated solution efficient [3,10].

4.3 Constraints and soft constraints

- Actuator constraints: joint position, speed, torque/current, and command acceleration limits.
- Geometry constraints: reachable workspace, collision avoidance, and singularity margins.

- Contact constraints: no pulling on the floor, friction-cone limits, and zero force on swing feet.
- Safety constraints: body-angle envelope, minimum ground clearance, thermal budget, and solver deadlines.

A hard constraint makes any violation infeasible. A soft constraint introduces nonnegative slack epsilon and penalizes it heavily:

$$Cx \leq d + \epsilon, \epsilon \geq 0, J_{soft} = J + \rho\|\epsilon\|_1 \quad (4.7)$$

Softening selected performance constraints prevents numerical failure when reality briefly violates a model assumption. Truly dangerous limits should remain hard or be enforced by an independent safety layer.

4.4 Recursive feasibility and stability

A standard stability design chooses terminal cost Vf(x)=x^TPx and terminal set Xf such that a local feedback kappa_f keeps Xf invariant and decreases the terminal cost:

$$f(x, \kappa_f(x)) \in \mathcal{X}_f, V_f(f(x, \kappa_f(x))) - V_f(x) \leq -\ell(x, \kappa_f(x)) \quad (4.8)$$

> **Shifted-sequence proof idea.** Suppose the optimal sequence at time k is feasible. At k+1, discard its first input, shift the remaining inputs left, and append the terminal controller kappa_f. Invariance makes this candidate feasible. The terminal decrease condition makes its cost smaller by at least the stage cost. Since the new optimum is no worse than this candidate, the optimal value decreases and acts as a Lyapunov function.

This theorem requires accurate assumptions, a feasible initial condition, correct terminal ingredients, and successful optimization. A practical robot also needs a fallback command if the solver times out or returns infeasible.

4.5 Nonlinear MPC

Nonlinear MPC replaces Ax+Bu with nonlinear dynamics f(x,u). The problem is generally nonconvex. Sequential quadratic programming, differential dynamic programming, iterative LQR, or direct multiple shooting solve local approximations. They can model rigid-body and contact dynamics more accurately, but computation and initialization become more demanding.

4.6 Centroidal quadruped dynamics

A common reduced model treats the trunk as one rigid body and legs as force-producing contacts. Let m be mass, p the center of mass, R the body orientation, omega angular velocity, ri the vector from COM to foot i, and fi its ground reaction force. Then

$$m\ddot{p} = \sum_{i=1}^{4} f_i + mg \quad (4.9)$$

$$I\dot{\omega} + \omega \times I\omega = \sum_{i=1}^{4} r_i \times f_i \quad (4.10)$$

For a stance foot on a flat surface, a polyhedral approximation to Coulomb friction is

$$f_{z,i} \geq 0, |f_{x,i}| \leq \mu f_{z,i}, |f_{y,i}| \leq \mu f_{z,i} \quad (4.11)$$

Torque-controlled quadruped MPC usually optimizes fi and maps them to joint torques tau=J^T f. MIT Cheetah work uses this structure because direct torque actuation makes ground-reaction-force commands meaningful [7,8].

4.7 MPC with your position-controlled actuators

Your current firmware exposes position PD rather than an MIT-style position-velocity-torque feedforward command. MPC is still possible, but its input should be something the closed-loop actuator can reliably realize, such as body pose offsets, foot-placement offsets, phase-speed corrections, or joint-position increments.

$$x_{k+1} = f_{\text{cl}}(x_k, \Delta p_{\text{foot},k}), \Delta p_{\text{foot},k} = u_k \quad (4.12)$$

Here fcl is the measured closed-loop robot including position controllers, gearboxes, backlash, and contact. It can be identified from small safe perturbation experiments. A position-MPC objective could penalize body attitude, angular rate, commanded offset, and command change:

$$J = \sum_{i=0}^{N-1} \left(\|\eta_i - \eta_r\|_{Q_\eta}^2 + \|\omega_i\|_{Q_\omega}^2 + \|u_i\|_R^2 + \|u_i - u_{i-1}\|_S^2 \right) \quad (4.13)$$

> **Honest limitation.** Position MPC can improve posture and foot placement, but it does not directly allocate contact forces. The inner PD loops and ground determine the actual forces. Once torque feedforward and reliable contact sensing exist, force-based MPC becomes much closer to the standard quadruped literature.

CHAPTER 5

Machine learning

Function approximation from data

5.1 What is learned?

Machine learning selects a function from a parameterized family using data. Given examples D={(xi,yi)} from an unknown distribution, supervised learning minimizes empirical risk

$$\hat{R}(\theta) = \frac{1}{n}\sum_{i=1}^{n} \ell(f_\theta(x_i), y_i) \quad (5.1)$$

The real goal is expected risk R(theta)=E[ell(f_theta(X),Y)] on future data. A model can reduce training loss by memorization without learning the underlying relationship. Generalization is the gap between empirical and expected performance.

5.2 Learning paradigms

Table 5.1. Learning signals relevant to a quadruped.

Paradigm	Data signal	Robot example
Supervised	input-label pairs	predict foot contact or actuator temperature
Self-supervised	labels constructed from data	predict next IMU/encoder state
Unsupervised	structure without labels	cluster terrain or actuator behavior
Reinforcement learning	delayed scalar reward	learn velocity-tracking locomotion policy
System identification	input-output trajectories	fit closed-loop A and B for LQR/MPC

5.3 Gradient descent

If the loss is differentiable, gradient descent updates parameters opposite the local direction of steepest increase:

$$\theta_{k+1} = \theta_k - \alpha \nabla_\theta \hat{R}(\theta_k) \quad (5.2)$$

For an L-smooth convex function, the descent lemma states

$$f(y) \le f(x) + \nabla f(x)^T (y - x) + \frac{L}{2}\|y - x\|^2 \quad (5.3)$$

Choose y=x-alpha grad f(x). If 0<alpha<2/L, then

$$f(x_{k+1}) \le f(x_k) - \alpha\left(1 - \frac{\alpha L}{2}\right)\|\nabla f(x_k)\|^2 \quad (5.4)$$

Proof. Substitute the gradient step into the descent lemma. The linear term becomes -alpha||grad f||^2 and the quadratic term becomes (L alpha^2/2)||grad f||^2. Their sum is negative under the step-size condition. Neural-network losses are nonconvex, so this proof does not guarantee a global optimum, but it explains the role of learning rate locally.

5.4 Bias, variance, and regularization

For squared-error regression, expected prediction error can be decomposed conceptually into irreducible noise, squared bias, and variance. A small model may systematically miss the dynamics; a large model may fit sensor noise and simulation artifacts. Regularization adds a preference such as small weights:

$$\min_{\theta} \hat{R}(\theta) + \lambda \|\theta\|_2^2 \quad (5.5)$$

For robot learning, data diversity is often more important than network size. Domain randomization, latency randomization, sensor noise, friction variation, motor-strength variation, and mass/COM variation teach a policy not to depend on one perfect simulator.

5.5 System identification as the first ML task

Before training a locomotion policy, estimate a local model from safe experiments. With collected transitions, fit A and B by least squares:

$$\min_{A,B} \sum_k \| x_{k+1} - Ax_k - Bu_k \|_2^2 \quad (5.6)$$

This modest use of learning directly supports LQR and MPC, reveals delays and coupling, and provides a reality check for simulation. It is usually a better first step than asking PPO to discover both physics and behavior simultaneously.

CHAPTER 6

Neural networks

Nonlinear function approximation and backpropagation

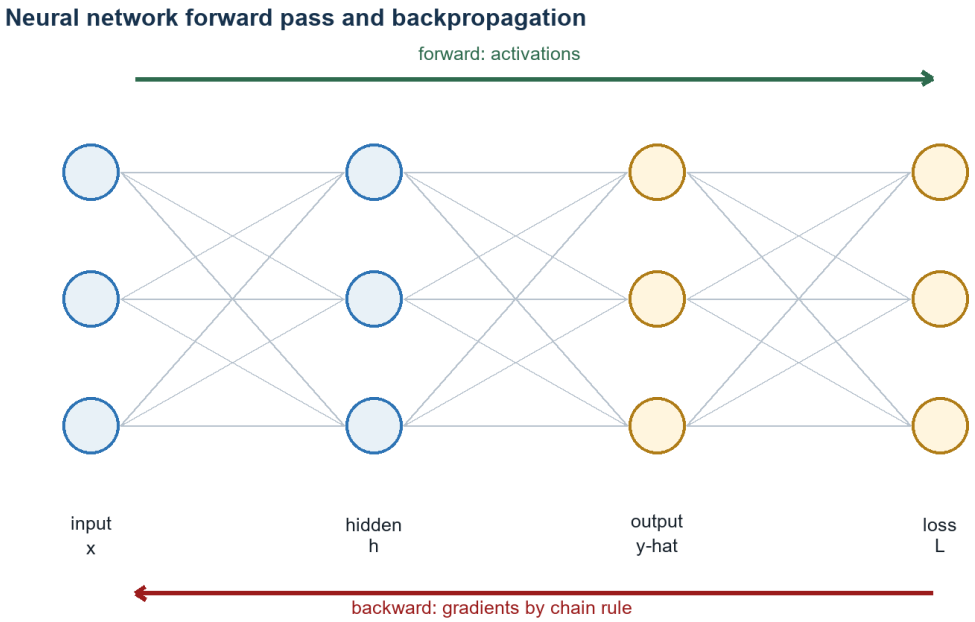


Figure 6.1. Forward propagation computes predictions; backpropagation computes every parameter gradient efficiently.

6.1 A neuron and a multilayer network

A neuron forms an affine combination followed by a nonlinear activation:

$$z = w^T x + b, a = \sigma(z) \quad (6.1)$$

For layer l in a feedforward network

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, a^{(\ell)} = \sigma^{(\ell)}(z^{(\ell)}) \quad (6.2)$$

Without nonlinear activations, composing layers collapses to one affine map. ReLU sigma(z)=max(0,z), tanh, and sigmoid provide nonlinear representational power. For continuous motor actions, the policy often outputs the mean of a Gaussian and uses tanh or explicit clipping to keep actions bounded.

6.2 Universal approximation: what it says and does not say

A classical universal-approximation theorem states that finite sums of sigmoidal ridge functions can approximate any continuous function on a compact domain arbitrarily well [5]. In simplified form, for every continuous f on a compact set K and epsilon>0, there exist N, coefficients alpha_i, vectors w_i, and biases b_i such that

$$\sup_{x \in K} |f(x) - \sum_{i=1}^{N} \alpha_i \, \sigma(w_i^T x + b_i)| < \epsilon \quad (6.3)$$

> **Important boundary.** The theorem is an existence statement. It does not say gradient descent will find those parameters, that the required network is small, that finite data identify the right function, or that the approximation is safe outside the training domain.

6.3 Backpropagation derivation

For one sample with scalar loss L, define the layer error delta^(l)=partial L/partial z^(l). At the output layer, the chain rule gives

$$\delta^{(L)} = \nabla_{a^{(L)}} \mathcal{L} \odot \sigma'^{(L)}(z^{(L)}) \quad (6.4)$$

Because z^(l+1)=W^(l+1)a^(l)+b^(l), the hidden-layer recursion is

$$\delta^{(\ell)} = \left(W^{(\ell+1)T} \delta^{(\ell+1)}\right) \odot \sigma'^{(\ell)}(z^{(\ell)}) \quad (6.5)$$

Finally, each weight affects its layer preactivation linearly, so

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \delta^{(\ell)} (a^{(\ell-1)})^T, \frac{\partial \mathcal{L}}{\partial b^{(\ell)}} = \delta^{(\ell)} \quad (6.6)$$

> **Why backpropagation is efficient.** Naively differentiating the loss separately with respect to every weight repeats the same chain-rule products. Backprop stores intermediate activations in the forward pass and reuses a single reverse recursion. It is reverse-mode automatic differentiation specialized to a computation graph [6].

6.4 Losses and outputs

- Regression: mean squared error, useful for predicting torque, temperature, or next-state residual.
- Classification: cross-entropy, useful for contact/slip/fault classification.
- Policy mean: continuous joint command or reference residual.
- Value output: expected discounted return from the current state.

A Gaussian policy with diagonal covariance is often written

$$a \sim \pi_\theta(a \mid s) = \mathcal{N}(\mu_\theta(s), \mathrm{diag}(\sigma_\theta^2)) \quad (6.7)$$

6.5 Neural networks in this project

A neural network should solve a clearly defined estimation or residual problem, not replace known geometry without reason. High-value candidates are actuator residual modeling, contact probability, slip detection, terrain classification, learned state-estimator correction, and a residual policy added to a safe nominal trajectory.

$$u = u_{\text{nominal}} + \mathrm{clip}(\Delta u_\theta(s), -\Delta u_{\max}, \Delta u_{\max}) \quad (6.8)$$

> **Safe learning architecture.** Keep homing, CAN mapping, limits, watchdog, IK/Jacobian, and emergency shutdown deterministic. Let learning adjust a bounded reference layer first. This preserves the controller structure you can reason about while still allowing adaptation.

CHAPTER 7

Reinforcement learning

Markov decision processes, value functions, and policy gradients

7.1 The Markov decision process

An MDP consists of states S, actions A, transition law P(s'|s,a), reward r(s,a,s'), and discount gamma in [0,1). At time t, the agent observes st, samples at from policy pi, receives rt, and transitions to st+1. The discounted return is

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k} \quad (7.1)$$

The Markov property says the conditional distribution of the future depends on the present state and action, not the entire past. If the observation omits motor temperature, contact state, or body velocity, the problem is partially observed. History stacking or a recurrent policy can help, but state estimation is the cleaner engineering response when physics provides structure.

7.2 Value functions and Bellman equations

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid s_t = s] \quad (7.2)$$

$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}[G_t \mid s_t = s, a_t = a] \quad (7.3)$$

Split the return into the immediate reward and discounted future return:

$$V^{\pi}(s) = \mathbb{E}_{a\sim\pi, s'\sim P}[r(s,a,s') + \gamma V^{\pi}(s')] \quad (7.4)$$

The optimal value satisfies

$$V^{\star}(s) = \max_{a} \mathbb{E}[r(s,a,s') + \gamma V^{\star}(s')] \quad (7.5)$$

7.3 Bellman contraction proof

Define the Bellman optimality operator (TV)(s)=max_a E[r+gamma V(s')]. For any bounded V and W, use the inequality |max f-max g|<=max|f-g|:

$$\|TV - TW\|_{\infty} \le \gamma \|V - W\|_{\infty} \quad (7.6)$$

Proof. For each state and action, the reward cancels and the expected value difference is at most gamma||V-W||∞. Taking the maximum over actions cannot increase that bound. Since gamma<1, T is a contraction. Banach's fixed-point theorem then gives a unique fixed point V*, and repeated Bellman updates converge to it in the tabular exact case.

7.4 Policy gradients

For trajectory tau=(s0,a0,...) with probability ptheta(tau), define J(theta)=E_tau[R(tau)]. Differentiate under the integral and use the log-derivative identity grad p=p grad log p:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}[R(\tau)\nabla_\theta \log p_\theta(\tau)] \quad (7.7)$$

Environment transitions do not depend on theta, so only policy probabilities remain:

$$\nabla_\theta \log p_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \quad (7.8)$$

Replacing full-trajectory return with return-to-go preserves expectation and reduces variance. Subtracting a state-only baseline also preserves expectation because

$$\mathbb{E}_{a\sim\pi}[\nabla_\theta \log \pi_\theta(a \mid s) b(s)] = b(s)\nabla_\theta \sum_a \pi_\theta(a \mid s) = 0 \quad (7.9)$$

Choosing b(s)=Vpi(s) produces the advantage A=Q-V, yielding the policy-gradient theorem in sampled form:

$$\nabla_\theta J(\theta) = \mathbb{E}[\nabla_\theta \log \pi_\theta(a_t \mid s_t) A^\pi(s_t, a_t)] \quad (7.10)$$

7.5 Actor-critic and GAE

The actor is the policy; the critic estimates V. A one-step temporal-difference residual is

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \quad (7.11)$$

Generalized Advantage Estimation forms an exponentially weighted sum

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{\ell=0}^{T-t-1} (\gamma\lambda)^\ell \delta_{t+\ell} \quad (7.12)$$

Small lambda relies more on the critic and has lower variance but more bias. Lambda near one approaches Monte Carlo returns, with lower bootstrap bias and higher variance.

7.6 Designing a quadruped MDP

A position-policy observation might include body orientation and angular velocity, estimated body velocity, 12 joint positions, 12 joint velocities, commanded planar velocity, gait phase, and previous action. Actions can be normalized residual joint-position targets. A representative reward is

$$r = w_v \exp\left(-\frac{\|v_{\mathrm{cmd}}\|^2}{\sigma_v^2}\right) + w_\omega \exp\left(-\frac{\|\omega_z - \omega_{\mathrm{cmd}}\|^2}{\sigma_\omega^2}\right) - w_\tau \|\tau\|^2 - w_{\Delta a}\|a_t - a_{t-1}\|^2 - w_{\mathrm{slip}} c_{\mathrm{slip}} \quad (7.13)$$

Every reward term creates an incentive. Penalizing torque too heavily can teach weak motion; penalizing action rate too weakly can create chatter; rewarding velocity without posture or contact constraints can teach falling or foot scraping. Terminations and reset distributions are part of the objective, not housekeeping.

CHAPTER 8

Proximal policy optimization

A practical on-policy actor-critic algorithm

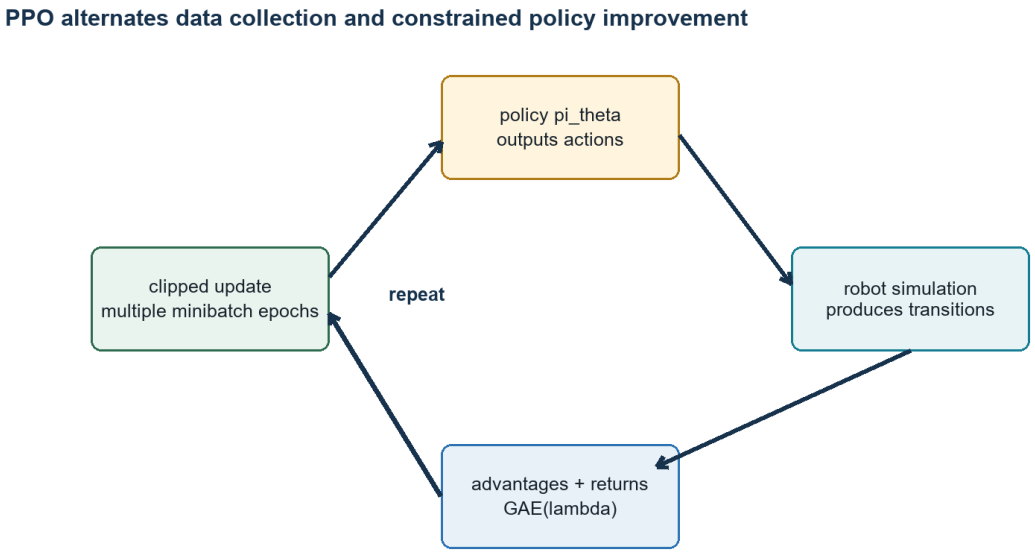


Figure 8.1. PPO repeatedly samples trajectories, estimates advantages, and updates the actor and critic.

8.1 Why policy updates need restraint

Policy-gradient samples are noisy. A large parameter step can make actions that generated the data unlikely under the new policy, invalidating the local gradient estimate and destroying a useful behavior. PPO uses the probability ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \quad (8.1)$$

The unclipped surrogate is E[r_t A_hat_t]. Importance sampling explains the ratio: data were drawn from the old policy, but the objective evaluates the new policy.

8.2 Clipped surrogate objective

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right] \quad (8.2)$$

If A_hat is positive, increasing the selected action's probability helps, but benefit is capped once r exceeds 1+epsilon. If A_hat is negative, decreasing its probability helps, but the objective blocks excessive decrease below 1-epsilon. The minimum selects the pessimistic objective in each case.

> **What clipping does not prove.** PPO clipping is a practical surrogate, not a universal monotonic-improvement theorem. The actual KL divergence can still become too large, especially over many

> epochs or with poorly scaled advantages. Monitor approximate KL, clip fraction, entropy, value loss, and explained variance [9].

8.3 Complete actor-critic loss

$$L(\theta, \phi) = L^{\text{CLIP}}(\theta) - c_v \mathbb{E}\left[(V_\phi(s_t) - \hat{R}_t)^2\right] + c_e \mathbb{E}[\mathcal{H}(\pi_\theta(\cdot|\, s_t))] \quad (8.3)$$

Implementation often minimizes the negative of this expression. The value term trains the critic; entropy discourages premature collapse to a nearly deterministic policy. Advantage normalization changes optimization scale but not the ordering of samples within a batch.

8.4 PPO algorithm

```
initialize actor parameters theta and critic parameters phi
repeat:
    collect T steps from many simulated robots with pi_theta_old
    compute returns and GAE advantages
    normalize advantages
    for several epochs:
        shuffle rollout into minibatches
        update actor with clipped surrogate
        update critic with value loss
        optionally stop early when KL exceeds threshold
    theta_old <- theta
```

8.5 Why PPO is used for locomotion

PPO is simple, parallelizable, and works with continuous actions. Large numbers of simulated robots can collect on-policy data. Successful sim-to-real quadruped work models actuator dynamics, latency, contact uncertainty, and random physical parameters; it does not merely train on a perfect nominal simulator [11].

For your robot, the first PPO policy should output bounded residual position targets at a moderate policy rate, while the deterministic C++ controller interpolates and enforces limits at 200 Hz. Training should start with standing and velocity tracking before rough terrain or recovery.

8.6 PPO safety and validation

- Train entirely in simulation until falls, action rates, joint limits, and thermal proxies meet acceptance criteria.
- Randomize mass, COM, friction, battery voltage, motor strength, backlash/dead zone, sensor bias, latency, and packet delay.
- Deploy on a stand first with action-scale limits and a deterministic nominal posture.
- Use a hardware watchdog, fall detector, current/temperature limits, and immediate IDLE path independent of the policy.
- Log observation, raw action, filtered action, joint command, feedback, reward terms, and every safety intervention.

CHAPTER 9

How the methods fit together

A hierarchy rather than a competition

Table 9.1. The methods occupy different layers.

Method	Model needed	Constraints	Online computation	Best role on your robot
PID/PD	little	saturation added manually	very low	joint servo and local posture loop
LQR	linear local model	not explicit	low	small-angle body regulation
MPC	predictive model	explicit	medium/high	foot/body reference planning
Neural net	data replaces explicit map	not inherent	low at inference	residual model or policy
PPO	simulator/experience	reward and safety wrapper	training high; inference low	learned locomotion references

There is a deep mathematical connection. LQR computes an exact value function for linear dynamics and quadratic cost. MPC numerically approximates the solution of a finite-horizon Bellman problem at every time step. Reinforcement learning approximates value functions or policies from sampled experience. A policy network can be viewed as amortized optimization: expensive training produces a fast function that maps state directly to action.

$$\text{MPC: } u_k = \underset{U}{\operatorname{argmin}}\, J(U; x_k), \text{Policy: } u_k \approx \pi_\theta(x_k) \quad (9.1)$$

A robust engineering architecture can combine them: PID at the motor, state estimation from sensors, MPC for constrained references, and a learned residual model or residual policy. The safety monitor remains outside all of them.

CHAPTER 10

Your current quadruped architecture

Where the mathematics enters the software you already built

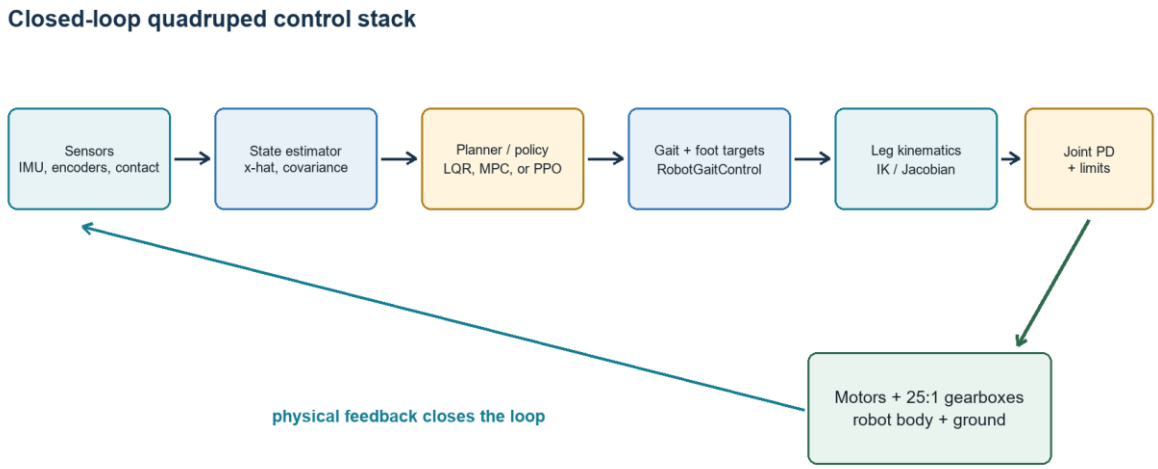


Figure 10.1. Recommended closed-loop hierarchy. The estimator is the missing bridge between raw IMU data and optimization.

10.1 Inspected Python snapshot

The current Python repository separates trajectory generation, gait scheduling, per-state gains, leg hardware mapping, and robot orchestration. At the inspected snapshot, trajectory_config.py produces Cartesian foot paths and IK angle deltas without owning CAN IDs or motor signs. gait_scheduler.py owns phase offsets. leg_config.py owns homing, geometry, neutral foot references, motor signs, CAN IDs, and the 25:1 gear ratio. gains_config.py owns startup, move, run, and hold gains.

Table 10.1. Current Python project snapshot; re-check values before testing because configuration evolves.

Current item	Inspected value	Control meaning
Python trajectory rate	80 Hz	host foot-target update rate
Neutral foot center	x=-70 mm, y=84.26 mm	rearward contact shift and shoulder offset
Paw radius	30 mm	linkage target is desired body height minus paw radius
Regular path	80 mm length, 90 mm height	open-loop planar stance/swing geometry
Stance ratio	0.80	fraction of phase assigned to stance
Vertical step	40 mm	march-in-place trajectory
Run gains	Kp=0.20, Kd=0.015, torque	current per-joint firmware commands

Current item	Inspected value	Control meaning
	limit=1.0	
IMU utility	BNO055 live readout	measurement only; no estimator or balance loop

10.2 Inspected C++ snapshot

The C++ stack runs a 200 Hz gait configuration, uses RobotGaitControl for robot-wide trajectory profiles and phases, and keeps Leg responsible for kinematics and joint commands. Leg::calculateTrajectoryCommand computes a desired foot target, desired foot velocity, a damped inverse Jacobian, joint velocity, and integrated joint-position command. This is a resolved-rate position controller:

$$\dot{q}_d = J^{\#}(q)\dot{x}_d + K_q(q_{\mathrm{IK}}(x_d) - q) \quad (10.1)$$

$$q_{\mathrm{cmd},k+1} = q_{\mathrm{cmd},k} + \mathrm{clamp}(\dot{q}_d)\Delta t \quad (10.2)$$

This controller is deterministic and geometrically meaningful. MPC or PPO should produce corrected xd, x-dot-d, gait phase, or bounded residual q targets above this layer rather than duplicating CAN and kinematic logic.

10.3 The IMU kinematic idea

For a point fixed in the body at vector r from the body origin, the velocity induced by body angular velocity omega is omega cross r. If the goal is to keep a world foot approximately fixed while the body rotates, the corrective foot velocity in the body description has the opposite sign:

$$v_{\mathrm{foot,corr}} = -\omega_{\mathrm{body,corr}} \times r_{\mathrm{foot}} \quad (10.3)$$

This cross-product relationship is kinematically valid. The controller still needs correct frame conventions, estimated orientation error, measured angular velocity, filtering, saturation, and an integration state that persists across loop calls. It should begin as a low-bandwidth posture correction, not as an unbounded double integration of noisy acceleration.

10.4 Why IMU alone is not a complete state estimator

Gyroscope integration drifts due to bias. Accelerometer measurements include gravity and specific force; recovering world acceleration requires an orientation estimate, and double integration amplifies bias into position drift. During stance, leg kinematics provide a powerful pseudo-measurement: a non-slipping foot has nearly zero world velocity.

$$v_{\mathrm{foot}}^{\mathrm{world}} = v_{\mathrm{body}}^{\mathrm{world}} + \omega^{\mathrm{world}} \times r_i^{\mathrm{world}} + J_i(q)\dot{q}_i \approx 0 \quad (10.4)$$

An EKF can fuse IMU propagation with stance-foot constraints. Contact probability determines measurement confidence. Even a simple complementary filter for orientation plus stance-constrained velocity is a major improvement over raw Euler angles.

CHAPTER 11

A staged implementation roadmap

From reliable standing to MPC and PPO without skipping foundations

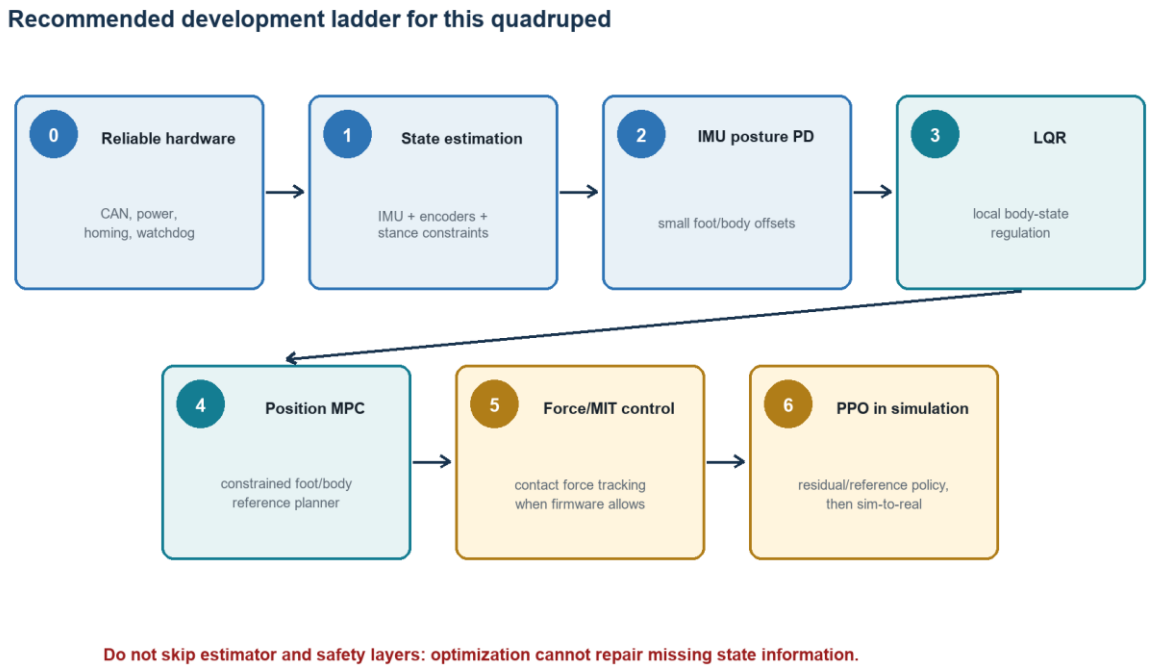


Figure 11.1. Each stage creates the evidence needed by the next stage.

11.1 Stage 0: preserve the deterministic baseline

1. Make all 12 motors ping reliably under load, not only at idle.
2. Validate homing, neutral pose, signs, gear ratio, limits, and emergency shutdown for every leg.
3. Log timestamps, commanded/measured position and velocity, current, bus voltage, mode, error flags, and dropped feedback.
4. Measure control-loop timing distribution and reject stale feedback explicitly.
5. Create stand tests and low-height ground tests with acceptance thresholds.

11.2 Stage 1: state estimator

Define one frame convention and timestamp every sensor. Start with orientation and angular velocity. Then estimate body velocity using stance-foot kinematics. Represent orientation internally with quaternions or rotation matrices; Euler angles may remain for telemetry.

$$\hat{x}_{k|k-1} = f(\hat{x}_{k-1|k-1}, u_{k-1}), P_{k|k-1} = F_k P_{k-1|k-1} F_k^T + Q_k \quad (11.1)$$

$$K_k = P^- H^T (H P^- H^T + R)^{-1}, \hat{x}^+ = \hat{x}^- + K(y - h(\hat{x}^-)) \quad (11.2)$$

11.3 Stage 2: low-bandwidth IMU posture PD

Before MPC, command small symmetric foot-height or body-angle offsets from roll/pitch error and angular velocity. Ramp gains from zero, clamp each correction, and freeze or decay correction when contact confidence is low.

$$\Delta p_{\text{foot}} = S_p K_p(\eta_d - \eta) - S_d K_d \omega \quad (11.3)$$

Test disturbances in four directions while standing. The expected result is not perfect rejection; it is correct sign, bounded action, reduced peak angle, and no excitation of joint oscillation.

11.4 Stage 3: identify and test LQR

Apply small pseudo-random or chirp foot-offset commands on a support stand and during low-height standing. Fit a discrete model for roll, pitch, and rates. Validate one-step and multi-step prediction on held-out data. Design LQR and compare it against hand-tuned IMU PD using the same disturbances.

11.5 Stage 4: position-based MPC

Run a small QP at 20-50 Hz while keeping the 200 Hz C++ leg loop. Let MPC output bounded corrections; interpolate between MPC solutions. A practical state and input are

$$x = [\phi, \theta, \dot{\phi}, \dot{\theta}, v_x, v_y]^T, u = [\Delta x_{\text{FL}}, \Delta z_{\text{FL}}, \dots, \Delta x_{\text{RR}}, \Delta z_{\text{RR}}]^T \quad (11.4)$$

Use phase-dependent constraints: swing feet can move to landing targets, stance feet receive small body-support corrections. If the QP misses its deadline, hold or decay the previous correction and continue the deterministic gait.

11.6 Stage 5: torque or MIT-style low-level mode

Only after the firmware supports safe torque feedforward and current/thermal limits should the optimizer command contact forces. Validate one actuator, one leg on a fixture, static whole-body force distribution, and slow crawl before dynamic trot. The mapping is

$$\tau_i = J_i(q_i)^T f_i + \tau_{\text{ff},i} + K_p(q_{d,i} - q_i) + K_d(\dot{q}_{d,i} - \dot{q}_i) \quad (11.5)$$

11.7 Stage 6: PPO

Build a MuJoCo model whose inertial properties, joint axes, limits, contact geometry, gearbox behavior, delay, and motor bandwidth are validated against logs. First train standing, then commanded velocity, then turning, then disturbances. Prefer a residual policy around a nominal controller for the first hardware deployment.

> **Raspberry Pi 5 role.** The Pi 5 can execute a compact neural policy and a small linear QP while running the 200 Hz deterministic controller, provided computation is profiled, cooling is adequate, threads are

> isolated sensibly, and every optimizer has a deadline fallback. Train PPO and perform large simulation batches on a desktop or GPU computer.

11.8 Acceptance tests before each escalation

Table 11.1. Controller maturity gates.

Gate	Pass criterion	Failure response
Timing	99.9th percentile below deadline	reduce logging/model size; isolate real-time loop
Estimator	bounded drift and correct disturbance sign	fix frames/calibration before control
Standing	repeatable 60 s hold with margin	retune or lower body before gait
MPC	feasible and deadline-safe under replay	fallback to LQR/zero correction
Simulation policy	randomized validation and no reward exploit	redesign reward/randomization
Hardware policy	stand test with bounded residual action	abort and inspect logs

CHAPTER 12

Worked quadruped examples

Turning equations into engineering calculations

12.1 From joint motion to foot velocity

For a three-DOF leg with joint vector q, forward kinematics gives foot position p=f(q). Differentiate with respect to time:

$$\dot{p} = \frac{\partial f}{\partial q}\dot{q} = J(q)\dot{q} \quad (12.1)$$

Near singularity, direct inversion amplifies noise. The damped least-squares inverse solves

$$\dot{q}^{\star} = \underset{\dot{q}}{\operatorname{argmin}} \left(\|J\dot{q} - \dot{p}_d\|^2 + \lambda^2 \|\dot{q}\|^2 \right) \quad (12.2)$$

Set the gradient to zero:

$$(J^T J + \lambda^2 I)\dot{q} = J^T \dot{p}_d \quad (12.3)$$

$$J_{\lambda}^{\#} = (J^T J + \lambda^2 I)^{-1} J^T \quad (12.4)$$

> **Meaning of damping.** The lambda term penalizes enormous joint speed. It trades exact Cartesian tracking for bounded motion near singularity. Adaptive damping can increase lambda as the smallest singular value of J decreases.

12.2 Static support-force distribution

For level static standing, the vertical forces sum to weight and their moments balance the COM. In a sagittal two-support approximation with front and rear contact distances lf and lr from the COM,

$$F_f + F_r = mg, F_f l_f = F_r l_r \quad (12.5)$$

$$F_f = mg\frac{l_r}{l_f + l_r}, F_r = mg\frac{l_f}{l_f + l_r} \quad (12.6)$$

A rear-biased COM increases rear normal load. Shifting both feet rearward relative to the body changes lf and lr and can redistribute support. But during dynamic gait, inertial moments and contact timing dominate the static calculation. MPC explicitly predicts these effects; an open-loop trajectory does not.

12.3 Why a foot can move but the body does not

Forward body acceleration requires a net forward ground-reaction force while stance contacts satisfy friction. If the swing foot lands with backward relative velocity, it produces a braking impulse. If stance feet slip, the intended body impulse is lost. If support forces are asymmetric, yaw appears. Therefore a visually correct foot loop is necessary but not sufficient.

$$m\Delta v_x = \int_{t_0}^{t_1} \sum_i f_{x,i}(t)dt \quad (12.7)$$

The integral is impulse. Position trajectories influence it indirectly through contact, compliance, and PD error. Force MPC or a learned policy can shape it more directly, but only when sensing and actuation support that objective.

12.4 A small position-MPC design example

Suppose system identification around standing yields a discrete roll model with sample period 0.025 s:

$$\begin{bmatrix} \phi_{k+1} \\ \dot{\phi}_{k+1} \end{bmatrix} = \begin{bmatrix} 1 & 0.025 \\ 0 & 0.96 \end{bmatrix} \begin{bmatrix} \phi_k \\ \dot{\phi}_k \end{bmatrix} + \begin{bmatrix} 0.002 \\ 0.16 \end{bmatrix} u_k \quad (12.8)$$

Let u be left-right differential foot-height correction in millimeters, constrained to |u|<=8 and |u_k-u_(k-1)|<=1. Choose Q=diag(100,8), R=0.2, horizon N=12. This QP predicts 0.3 s ahead. The units of Q and R are not physically pure because they normalize unlike quantities; scale states and inputs so tuning remains interpretable.

> **Validation experiment.** Replay recorded IMU disturbances through the identified model and compare PID, LQR, and MPC. Then run the same bounded push test on the real robot. Compare peak roll, settling time, maximum correction, current, and missed deadlines. This turns controller choice into evidence rather than taste.

CHAPTER 13

Proofs, assumptions, and common misconceptions

What the mathematics guarantees and where the guarantees stop

13.1 PID is not automatically stable

Feedback can destabilize a plant when sign, gain, phase lag, delay, saturation, or unmodeled resonances are wrong. The word feedback describes an architecture; stability follows from closed-loop analysis.

13.2 LQR optimality is model-relative

LQR is globally optimal for the stated linear dynamics, quadratic cost, and unconstrained input. It is not globally optimal for the nonlinear contact robot. Q and R define the objective; changing them changes what optimal means.

13.3 MPC is not magic prediction

MPC predicts with a model. A poor model, wrong contact schedule, stale state, or infeasible constraints can produce poor decisions. Receding-horizon feedback corrects some mismatch but does not eliminate it.

13.4 Universal approximation is not universal safety

Representational capacity does not establish trainability, robustness, calibration, or behavior outside the compact training domain. A neural policy is a numerical function, not a proof of stability.

13.5 PPO clipping is not a hard action bound

The probability-ratio clip shapes the training objective. It does not directly clip physical joint targets. Action scaling, joint limits, rate limits, and watchdogs must be implemented separately.

13.6 More loop frequency is not always better

A higher nominal rate helps only if sensing, computation, communication, and actuation complete reliably. A 500 Hz loop with stale PDO data and frequent overruns can be worse than a deterministic 200 Hz loop. Analyze latency and jitter as part of the dynamics.

CHAPTER 14

Exercises

Problems for building real understanding

14.1 Control theory

1. Derive the closed-loop poles of J q-ddot = tau under PD control. For J=0.08 kg m^2, choose Kp and Kd for natural frequency 12 rad/s and damping ratio 0.8.
2. Discretize a continuous double integrator with zero-order hold. Construct its controllability matrix and prove controllability for Delta t>0.
3. Carry out one complete finite-horizon LQR backward recursion for a scalar system x+=1.1x+u with Q=1, R=0.2, QN=2, and N=2.
4. Write the condensed MPC matrices Acal and Bcal for horizon N=3. Add input magnitude and slew-rate constraints.
5. Using a support polygon, derive the static front/rear normal loads as the COM moves 30 mm rearward.

14.2 Machine learning and PPO

1. For a two-layer scalar neural network, compute every derivative by hand and verify Equation 6.5.
2. Prove that subtracting any state-only baseline leaves the expected policy gradient unchanged.
3. Compute GAE for a five-step trajectory under lambda=0, 0.95, and 1. Compare bias and variance qualitatively.
4. For positive and negative advantages, graph the PPO clipped objective as a function of probability ratio r.
5. Design a quadruped reward and list at least five ways the policy could exploit it. Add terms or terminations to remove each exploit.

14.3 Project laboratories

1. Build a timestamped standing log and estimate the effective delay from commanded joint position to measured motion.
2. Perform small input perturbations and fit a roll/pitch linear model. Report training and validation prediction error.
3. Compare IMU PD and LQR under matched push tests. Use confidence intervals over repeated trials.
4. Run a position-MPC controller on recorded telemetry before enabling hardware output.
5. Create a MuJoCo parameter-randomization table whose ranges are justified by measurements from the physical robot.

14.4 Selected hints

Exercise 1. Match Js^2+Kd s+Kp to J(s^2+2 zeta omega_n s+omega_n^2): Kp=J omega_n^2 and Kd=2J zeta omega_n.

Exercise 3. Start with P2=QN. Compute K1 and P1, then K0 and P0 using Equations 3.7-3.8.

PPO graph. For A>0, the objective becomes flat above 1+epsilon. For A<0, it becomes flat below 1-epsilon. Carefully account for multiplication by a negative advantage before taking the minimum.

CHAPTER 15

Glossary and reference equations

A compact map for future coding

Table 15.1. Core vocabulary.

Term	Precise meaning
State	Sufficient variables for predicting future dynamics under known inputs.
Feedback	Action computed using measured or estimated system behavior.
Policy	A deterministic or stochastic mapping from state/observation to action.
Value function	Expected future return or optimal cost-to-go from a state.
Lyapunov function	Positive scalar function that decreases along closed-loop trajectories.
Riccati equation	Matrix equation defining the quadratic value function in LQR.
Receding horizon	Solve a future plan repeatedly but apply only the first action.
Jacobian	Local derivative mapping joint rates to Cartesian rates.
Singularity	Configuration where the Jacobian loses rank and some Cartesian motions require unbounded joint speed.
Actor-critic	Policy model paired with a learned value estimator.
On-policy	Data are collected using the current or nearly current policy.
Domain randomization	Training over varied simulated physics to improve robustness and transfer.

15.1 Equation map

- PID: Equation 2.1; discrete implementation: Equations 2.8-2.9.
- LQR gain and Riccati recursion: Equations 3.7-3.10.
- MPC optimization and condensed QP: Equations 4.1-4.6.
- Backpropagation: Equations 6.4-6.6.
- Bellman and policy gradient: Equations 7.4-7.10.
- GAE and PPO: Equations 7.12 and 8.1-8.3.
- Quadruped centroidal dynamics: Equations 4.9-4.11.
- Resolved-rate leg control: Equations 10.1-10.2 and 12.1-12.4.

CHAPTER 16

References

Primary papers, textbooks, and implementation references

[1] K. J. Astrom and R. M. Murray, Feedback Systems: An Introduction for Scientists and Engineers, 2nd ed. https://fbsbook.org

[2] S. Boyd, EE363 Linear Dynamical Systems: LQR, estimation, and Lyapunov lecture notes. https://ee363.stanford.edu/lectures.html

[3] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, Model Predictive Control: Theory, Computation, and Design, 2nd ed. https://sites.engineering.ucsb.edu/~jbraw/mpc/

[4] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. http://incompleteideas.net/book/the-book-2nd.html

[5] G. Cybenko, Approximation by Superpositions of a Sigmoidal Function, 1989. https://doi.org/10.1007/BF02551274

[6] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, Learning Representations by Back-Propagating Errors, Nature, 1986. https://doi.org/10.1038/323533a0

[7] J. Di Carlo et al., Dynamic Locomotion in the MIT Cheetah 3 Through Convex Model-Predictive Control, IROS 2018. https://dspace.mit.edu/bitstream/handle/1721.1/138000/convex_mpc_2fix.pdf

[8] D. Kim et al., Highly Dynamic Quadruped Locomotion via Whole-Body Impulse Control and MPC, 2019. https://arxiv.org/abs/1909.06586

[9] J. Schulman et al., Proximal Policy Optimization Algorithms, 2017. https://arxiv.org/abs/1707.06347

[10] B. Stellato et al., OSQP: An Operator Splitting Solver for Quadratic Programs. https://arxiv.org/abs/1711.08013

[11] J. Tan et al., Sim-to-Real: Learning Agile Locomotion for Quadruped Robots, 2018. https://arxiv.org/abs/1804.10332

[12] I. Dadiotis, A. Laurenzi, and N. Tsagarakis, Whole-Body MPC for a 37-DoF Legged Manipulator, 2023. https://arxiv.org/abs/2310.02907

[13] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning, MIT Press, 2016. https://www.deeplearningbook.org

[14] OSQP documentation, constrained linear MPC example. https://osqp.org/docs/examples/mpc.html

Closing perspective

The progression from PID to PPO is not a march from primitive to advanced. It is a progression in what information and computation are used. PID uses local error history. LQR uses a model and an infinite-horizon quadratic value function. MPC uses a model, current state, and explicit future constraints. PPO uses sampled experience to shape a nonlinear policy. A successful quadruped normally combines several of them, each at the layer where its assumptions are credible.

> **Recommended next intellectual milestone.** Implement and validate a state estimator, then compare IMU PD, LQR, and a small position-MPC controller on the same standing disturbance experiment. That single study will teach more control theory, modeling, statistics, and real-time engineering than jumping directly to a large learned gait policy.